Process Report – StudyHelper

Institution:
VIA University College

Course & Semester:
SEP4, 4th Semester

Authors (Student Numbers):
Alexandru Savin(354790)
Cristina Aurelia Matei (354776)
Damian Michal Choina (354789)
Eduard Fekete (355323)
Jakub Maciej Baczek (354814)
Karina Rubahova (354565)
Mara-Ioana Statie (354536)
Marta Zrno (355351)
Piotr Junosz (355502)
Tymoteusz Krzysztof Zydkiewicz (355413)

Supervisors:
Erland Ketil Larsen
Joseph Chukwudi Okika
Kasper Knop Rasmussen
Markéta Tranberg

Date:
May 25, 2026

Character Count:
38,512 characters

TABLE OF CONTENTS

1. INTRODUCTION

Organizationally, the process was managed by following Scrum-like sprints and utilizing the task tracker Jira. The sequence of steps undertaken included development of the initial frontend structure and authentication, routing, integrating, improving usability, expanding the backend and security, and ultimately testing, preparing for deployment, and documenting the process. Sprint planning, sprint review, and retrospective activities helped assess progress and manage the process between sprints Throughout the project, certain technical and collaboration challenges arose. Some implementation details were subject to change, which was expected given the nature of the process, as the degree to which certain issues needed to be addressed was reconsidered, including frontend and backend integration, user authentication, device-server connection, and the features to be implemented. The project utilized Scrum sprints and the Jira task tracking system. The project included the following tasks: creating the interface structure and authentication system, developing routing and integrating functionality, improving usability, adding backend and security features, testing, deployment, and documentation. Sprint planning, sprint reviews, and retrospectives were used to evaluate progress and adjust work between sprints. During the project, the team encountered a number of challenges, including both technical and collaboration-related ones. Some aspects of the project turned out to be different than expected, particularly in relation to frontend and backend integration, user authentication, device connectivity, and the implementation of specific features. # 2. Group Work

2.1 Group Composition and Profiles

[Introduce each group member briefly — background, relevant skills, and prior experience. If the group is multicultural or interdisciplinary, highlight this and reflect on how it influenced the collaboration.]

Member	Main Area	Main Contributions
Karina Rubahova	Frontend / Scrum	Localization, dashboard styling, theme system, testing, sprint documentation

Member	Main Area	Main Contributions
Cristina Matei	Frontend / Backend Integration	Authentication integration, responsive improvements, profile functionality, backend/frontend synchronization
Marta Zrno	Backend	JWT authentication, API endpoints, database-related functionality
Eduard Fekete		
Damian Michal Choina		
Piotr Junosz		
Jakub Maciej Baczek		
Mara-Ioana Statie		
Alexandru Savin		
Tymoteusz Krzysztof Zydkiewicz		

2.2 Roles and Contributions

The distribution of responsibilities was primarily based on technology areas: frontend, backend, MAL, and IoT. While initial plans for the distribution of responsibilities were developed early in the sprint planning process, the actual distribution gradually evolved throughout the semester as the project took shape. Frontend-related tasks included dashboard development, localization, theming, profile features, responsiveness, calendar support, testing, and deployment preparation. Backend tasks focused on developing the authentication system, API endpoints, JWTs, and database interactions. MAL tasks included model-based predictions, dataset generation, and fitness prediction experiments. Tasks performed by the IoT team focused on sensor integration and connectivity. All tasks were managed using Jira and GitHub. Jira was responsible for tracking sprint-related issues and their progress, while GitHub branches and pull requests were used for implementation-related work.

2.3 Team Dynamics and Collaboration

Communication took place via Discord, Jira, GitHub, and weekly project meetings. Discord was used for day-to-day communication, promptly exchanging requests, coordinating subgroup work, and sharing progress updates. Jira helped manage sprints, assign responsibilities, track progress, and prioritize tasks throughout the semester. It's important to point that GitHub played a significant role in facilitating collaboration across the various project components, namely the frontend, MAL, and IoT implementations. Functions were isolated using branches and pull requests to reduce the likelihood of repository conflicts when multiple users were present at the same time. Coordination between subteams was essential when implementing shared branches that were dependent on each other. The decision-making process within our group project relied heavily on technical responsibility and subject matter expertise. Front-end decisions were made by front-end developers, while MAL and IoT decisions were overseen by the relevant subgroups. Broader decisions regarding architecture or processes were discussed and adopted collectively during meetings. Over time, communication within the project became more organized as integration issues became more complex due to the interactions between the front-end, back-end, MAL and IoT layers. Specifically, API conventions, authentication processes, and device/session synchronization posed challenges for coordinated work.

2.4 Conflict and Resolution

During the project, the team encountered some technical disagreements and collaboration issues. Sometimes, these issues arose due to inconsistencies in requirements between the frontend and backend. This could lead to duplication of effort, changes to the previously chosen structure, and the need for additional effort to integrate all components. Furthermore, during one of the sprints described in the documentation, an issue arose: one team member left the team and was absent for subsequent sprints, which impacted task distribution and presentation preparation. These issues were largely resolved through improved communication, discussions during sprints, the use of Jira for task management, and a clearer assignment of roles within the team. Team members began to more carefully consider architectural and integration decisions before collaborating on any changes. One lesson learned was the need for clear assignment of roles and improved communication in large teams consisting of multiple subteams.

2.5 Social Loafing and Accountability

One of the challenges the group faced was ensuring equal contributions from each member to a large project throughout the semester. Because the project encompassed various areas—frontend, backend, MAL, and IoT—the amount of each member's contribution varied and was sometimes difficult to estimate, especially given that some tasks were more complex than others. The group employed multiple strategies to ensure greater transparency, such as using Jira for task assignments, GitHub for commits and pull requests, sprints and sprint reviews, and direct communication between team members. Task assignments were clear and continually updated during sprints. As the semester progressed, team members realized that problems could arise ddue to unclear responsibilities or poor communication, leading some team members to take on most of the integration and coordination work, unaware that they had been assigned other roles.This led to a greater focus in subsequent sprints on task allocation and decision-making before implementation began. This project clearly demonstrates that responsibility encompasses more than just task execution; it also encompasses communication, coordination, integration, and participation in meetings and sprints.

3. PROJECT INITIATION

3.1 Choosing the Problem Domain

[Why did your group choose this particular topic? Was it driven by personal interest, practical relevance, available resources, or suggestions from a supervisor/company? In hindsight, was this a good choice? What would you do differently?]

3.2 Problem Statement Development

[How did you arrive at the final problem statement? Describe the iterations. Was the initial framing too broad, too narrow, or off-target? What clarified your thinking — literature, supervisor input, prototyping?]

3.3 Time Planning and Scheduling

[How did you plan the project timeline? Did you use sprints, Gantt charts, or a looser milestone structure? How realistic was the initial plan? Reflect on deviations: what caused delays, and how were they managed?]

3.4 Ethical Perspectives

[What ethical questions arise from the problem your project addresses? Consider: data privacy, environmental impact, fairness, accessibility, or societal effects. Were these considerations built into the project from the start, or addressed reactively?]

4. PROJECT EXECUTION

This section describes the development process, divided into our collective efforts and team-specific contributions.

4.1 Together Process

This section describes the development process, divided into our collective efforts and team-specific contributions.

4.1 Together Process

As our project plan progressed, our system began to evolve into an integrated system requiring a high degree of interdependence across all components. Initially, at the beginning of the semester, subgroups were able to work more independently, as many components were implemented as mockupsor were in a work-in-progress state. However, as the development process progressed, the need for coordination increased significantly. First, the frontend depended on several aspects of the backend, including

authentication, API design, sessions, and sensor endpoints. Second, both the backend and MAL implementations relied on certain common conventions, such as JSON structures and predictive models. Thus, API contracts and data structures became integral to cross-team collaboration. To avoid collisions and conflicting implementations, the project relied heavily on branches and pull requests on GitHub. We used feature branches to allow teams to implement their solutions in parallel and then merge them into shared branches. However, the synchronization process sometimes became problematic due to changes in the backend and work on the frontend. Communication took place through Discord chats,sprint sessions, Jira tickets, and personal contacts with those responsible. The more complex the integration task, the clearer it became that even minor changes could simultaneously impact multiple aspects of the system.Among other interesting process observations, it's worth noting that the integration proved quite labor-intensive. Although everything worked individually, integrating all components, including the frontend, MAL, and IoT, proved to be a more complex task.

4.2 IOT Team Process

[... Describe the development of the embedded C code, hardware assembly, and testing. ...]

4.3 MAL Team Process

The Machine Learning and API (MAL) development followed an iterative path from data exploration to pipeline implementation. The following notes capture the key decisions and observations made during this process.

4.3.1 Mock Data Search and Goal Refinement

Initially, the focus was on how environmental noise influences focus. We attempted to combine datasets of focus-related noise effects with labeled sound categories from WAV files, extracting frequencies and loudness. However, we realized that "focus" cannot be measured directly. Consequently, we narrowed the goal to predicting a user-provided **Study Suitability Rating**, which made the target variable more grounded in user feedback.

4.3.2 Data Quality and Correlation Analysis

Further investigation led to the elimination of several datasets that appeared synthetic. We observed that in some candidate datasets, the distributions of features like humidity, noise, and light were suspiciously uniform, suggesting algorithmic generation rather than real sensor collection.

We also analyzed feature correlations as part of our validation process. Healthy datasets showed natural physical correlations (e.g., CO2, temperature, and humidity), while suspicious datasets exhibited either zero correlation or extreme overfitting potential. We decided to merge diverse datasets to create a more robust mock dataset.

4.3.3 Advanced Imputation Strategy

To handle missing values after merging, we implemented a sophisticated approach using the **MICE (Multivariate Imputation by Chained Equations)** framework. We enhanced this by:

- **Clustering**: Using k-means to find environment types (e.g., sessions made with high sun exposure vs. sessions made in labs).
- **ExtraTrees Estimator**: Modeling complex non-linear relationships.
- **Variance Modification**: Including natural distribution variance to avoid "flat average" imputation.
- **Global Median Fallback**: Used for extreme sparsity to prevent bias.

4.4 Frontend Team Process

The frontend development process changed significantly throughout the semester because the frontend depended heavily on MAL, and IoT integration. Many frontend decisions that seemed simple at the beginning became more complex once real API responses and session logic were introduced.

4.4.1 Initial Frontend Structure and Routing

In the early stages of the project, the frontend team focused on creating the core structure of the application in React. Initially, the developed modules focused on routing, navigation, login/registration screens, layout templates, and dashboard placeholders.

One of the key early decisions regarding the frontend was to divide the application into pages, components, services, and contexts to avoid bundling all the logic into large React components.

At this point, all frontend-related functions relied on mock data, as the backend API had not yet been fully developed.

4.4.2 Dashboard Development and UI Iterations

The dashboard emerged as the most frequently changed feature on the frontend throughout the semester. Initially, the dashboard was just filled with dummy cards and static information, however, further iterations of the dashboard included:

- Dynamic sensor values
- Recommendation cards
- Session-related information
- Prediction-related UI
- Responsive layouts
- Better loading and empty states

There were several parts which were reworked multiple times as there were changes in backend response and data structures when integrating. Some UI states which appeared to be straightforward at first became more complex after adding real data handling functionality.

The front end components had to support:

- Missing values
- Delayed responses
- Loading states
- Session synchronization
- Empty datasets
- Changing API structures

This showed that frontend complexity increased significantly once mocked data was replaced with real integration.

4.4.3 Localization and Theme System

Localization support was introduced during later frontend iterations. The application implemented English and Danish translations using React Context and centralized translation objects.

At the same time, a dark/light theme system was added using ThemeContext and localStorage persistence. Initially, theme handling looked relatively simple, but later many components needed additional refactoring because cards, charts, navigation, popups, and recommendation elements all needed to react dynamically to theme changes.

One problem discovered during this phase was that UI consistency became harder to maintain once localization and themes affected nearly every component in the system.

4.4.4 Backend Integration Challenges

4.4.5 Scrum Workflow and Task Distribution

A sprint-based methodology was used during frontend development, leveraging the Jira task management system. Tasks were broken down according to the requirements of a specific sprint and the needs of the frontend at that stage of the process.

Planning served as a guideline for determining the implementation order, and a review and retrospective process was used to assess any integration issues, UI issues, and unfinished work on the frontend.

Given that frontend processes were heavily dependent on backend processes, IOT and MAL, priorities could shift during implementation. Therefore, constant coordination between frontend and backend specialists was crucial.

4.4.6 Frontend Testing and Stabilization

Interface testing was only implemented during the final sprint due to integration difficulties. Tools such as Vitest, jsdom, React Testing Library, and the user-event Testing Library were used for interface testing.

These tests primarily covered:

- Dashboard rendering
- Theme switching
- Localization behavior
- localStorage persistence
- Session rating flow
- User interaction behavior

An important observation made during testing is that automated interaction tests helped identify frontend issues that would not have been detected by manual testing.

Overall, the frontend development phase demonstrated how frontend implementation relies heavily on communication and API quality, as well as the collaboration of various subteams.

4.5 Application of Methods and Theories

[Reflect on the engineering and analytical methods you applied (requirements analysis, architecture design, testing frameworks, etc.). Were the methods well-suited to the problem? Did you feel confident using them, or were there learning curves? What worked well and what fell short?]

4.6 Challenges During Execution

[Describe the most significant technical or organisational challenges encountered. How did the team respond? What was the outcome? Examples: “The I2C sensor driver took much longer than expected,” or “The API contract changes caused significant rework in the frontend.”]

4.7 Deviations from the Plan

[Compare what was planned versus what was actually executed. What changed and why? Were changes proactive (intentional pivots) or reactive (forced by circumstances)? How did the team adapt?]

4.8 Use of Tools and Technologies

[Reflect on the tools and technologies used throughout the project (IDEs, version control, project management, testing frameworks, etc.). Were they effective? Were there tools you wished you had used, or tools you regret choosing?]

5. PERSONAL REFLECTIONS

Piotr Junosz

Fourth semester work was so far the most challanging in terms of communication, expectations, performing and coordination within the big 11 people group. It was my first time trying to develop a solid project in such a big team consisting of 3 subteams, trying to make sense out of the important assigment.

The first significant change I noticed was how the motivation was working. It is well described by Victor Vroom's book "Work and motivation" where he says that effort is tied up to expectations (Vroom, 1964). So if you believe your hard work will result in a finished project, you are highly motivated, but if you look around and see that other people are not working, your expectation of success drops, and your motivation completely collapses. This was seen during this semester project period and led to periods of members not motivating themself as much as in previous semesters. Personally, this made the process of developing the system a lot less exciting.

Another correlated phenomenon that occured is called diffusion of responsibility where a person is less likely to take responsibility for an action when others are present (Darley & Latane, 1968). This has grown even more because the group had 11 people and it is easier to "hide" in this group than group consisting of 4-5 people. The larger a group becomes, the less individual work each member will take. This combined with problem of motivation made me realised few times that I have to work more as individual in order our project to succeed but then I understood that alone I cannot finish it fully which made me loose motivation and not enjoy the process of work on this project.

Having experienced those problems, I think what makes a big difference between a real job team and what i call "company simulation" of our big group is the role of a manager or a leader. Even though we were using Scrum roles such as Product owner and Scrum master, we were missing someone whos only responsibility would be focusing on connecting subteams work and generally leading the whole project process. I believe this could be a solution to at least the problem with motivation.

Karina Rubahova

At the beginning, I was actually interested about this project because it looked much bigger and more realistic than to previous semester projects. Earlier projects were usually smaller and easier, but this

time we had frontend, backend, MAL, IoT, authentication, predictions, sessions, and many connected parts working together. It was interesting, but also stressful. In my opinion, the most challenging in project was that we had to work in a large group. Because the entire group was divided into several subgroups, it sometimes felt as if each of them was acting as an independent unit rather than part of the main team.

One of the biggest technical challenges for me was that backend functionality and API structures were changing quite often. Sometimes frontend work was already finished, but later had to be changad again because authentication flow, JSON responses, or backend logic changed. Merge conflicts also became stressful sometimes, especially during later sprints when more people worked on connected functionality at the same time.

At the same time, I think this project helped me learn realistic software development. I improved my understanding of React architecture, Git workflow, large project structure, frontend/backend integration, and automated testing. Before this project, I did not really understand how important frontend testing can become in larger systems. Working with Vitest and React Testing Library helped me understand how tests can detect problems that are difficult to notice manually.

I think one of my stronger contributions during the project was helping keep frontend work moving forward even during unstable integration periods. I worked a lot on frontend testing, sprint documentation, localization, theme-related functionality, and frontend fixes during integration. I felt that my work was important and visible in the final system.

Even though the project was stressful at times, I still think it was a valuable experience. I learned much more about communication, integration, coordination, and realistic software development than in previous semester projects. The project also showed me that large systems are not only about writing code — a lot of the work is connected to communication, synchronization between teams and adapting to changes during development.

Piotr Junosz

Fourth semester work was so far the most challanging in terms of communication, expectations, performing and coordination within the big 11 people group. It was my first time trying to develop a solid project in such a big team consisting of 3 subteams, trying to make sense out of the important assigment.

The first significant change I noticed was how the motivation was working. It is well described by Victor Vroom's book "Work and motivation" where he says that effort is tied up to expectations (Vroom, 1964). So if you believe your hard work will result in a finished project, you are highly motivated, but if you

look around and see that other people are not working, your expectation of success drops, and your motivation completely collapses. This was seen during this semester project period and led to periods of members not motivating themself as much as in previous semesters. Personally, this made the process of developing the system a lot less exciting.

Another correlated phenomenon that occured is called diffusion of responsibility where a person is less likely to take responsibility for an action when others are present (Darley & Latane, 1968). This has grown even more because the group had 11 people and it is easier to “hide” in this group than group consisting of 4-5 people. The larger a group becomes, the less individual work each member will take. This combined with problem of motivation made me realised few times that I have to work more as individual in order our project to succeed but then I understood that alone I cannot finish it fully which made me loose motivation and not enjoy the process of work on this project.

Having experienced those problems, I think what makes a big difference between a real job team and what i call “company simulation” of our big group is the role of a manager or a leader. Even though we were using Scrum roles such as Product owner and Scrum master, we were missing someone whos only responsibility would be focusing on connecting subteams work and generally leading the whole project process. I believe this could be a solution to at least the problem with motivation.

Damian Michal Choina

This project was my first time writing embedded systems code in C, and the learning curve at the start was real. Getting used to manual memory management, configuring hardware registers by hand, and thinking about timing and interrupts in a way I never had to before took some adjustment. However, once the initial unfamiliarity wore off the work became genuinely enjoyable. I also had no previous experience setting up CI/CD at this scale. Building a GitHub Actions pipeline that automatically compiles the firmware, runs the full test suite, and uploads coverage reports on every pull request was something entirely new to me, and seeing it catch issues before they reached the hardware made the investment feel immediately worthwhile.

What made the technical challenges manageable was the sub-team dynamic. Because we already knew each other from previous SEP projects, we skipped the awkward early phase of figuring out how to work together and moved straight into productive collaboration. There was an existing level of trust that made it easy to ask for help, split work naturally, and review each other’s code without it feeling like criticism. Communication with the ML and Frontend teams was also smooth throughout. We agreed on API contracts early and checked in regularly enough that integration never became a crisis.

Looking back, the things I would do differently are straightforward. I would get a minimal end-to-end path working in the first week rather than building depth in one area before the pieces connect. I would write tests as the code is written rather than treating them as something to catch up on later. And I would set up the CI pipeline at the very start of the project, not once the codebase is already growing. The automation overhead feels expensive early on but pays back quickly once the project reaches any real complexity.

Eduard Fekete

Learning Outcomes

This take may sound paradoxical but I believe now in some power of pointless meetings. I believed that if there is no agenda, then there is no reason to meet. I can see that both supervisor meetings with merely explaining what we have done would be great early on and also some team meetings when we would just talk about what we have done, alternatively with some team-building aspect could have had a large impact.

I also learned that information should be structured in a more hierarchical way. Strongest project-shaping decisions should always be visible in a simple format, more details can be linked to that and so on... I often referenced details and technicalities by saying “there is a file in the repo” or “you can see it on Figma”, which in my head made sense, as I could fully orient in these environments but I could see towards the end that a large failure on my side as a Product Owner was that many people did not have a clear idea of what was where.

I also learned that there is a whole parallel universe going on besides school and work. It would be a nice utilitarianistic utopia to just let everyone deal with their own lives but it literally would improve collaboration to know what’s going on with people besides the project. Sometimes I just feel like an hour of talking and 2 hours of work could have been more efficient than even 4 hours of working.

Contribution and Role

I was the Product Owner of the project, which meant that I was responsible for defining the product vision, managing the backlog, and ensuring that the team was aligned with our goals. I took this role seriously and tried to be as proactive as possible in communicating with the team and the supervisors. I tried to develop and communicate a strong product vision since the beginning, maintaining many of the diagrams and overall documentation. On top of that, I often ended up being “the person” for people

who needed to quickly check on various decisions about to be made in the project, meaning I often found a lot of work even besides my typical work in my scrum team.

Although I believe that I have done my part in the project, I also feel that the way I approached everything could have been different. Different ways and channels of communicating all the information, better structuring of the information, and most importantly listening more to the team as often finding the best solution is not just about the problem itself but also about the team solving it.

I became a product owner mostly because this role stuck with me from previous projects and there was nobody else interested in it this time. A lot of trust and expectations were put on me, which I did my best to meet but I also can see that if I put more focus on the project, I could have achieved more. I remained my largest critic throughout the project, and I would love to have a peaceful mind concluding that I didn't get negative feedback, but how hypocritical would that be to say right after mentioning I didn't ask for enough feedback from the team?

Challenges and Growth

The most challenging aspect of the project was navigating the uncertainty of the situation overall. I can see many of my classmates, including myself, being affected by uncertainty and general confusion around the integration of artificial intelligence in learning and the industry in general. This affects motivation, and often prompts questioning of what is the point of our work and how should we approach getting help on such projects? How is it different talking with supervisors, searching around StackOverflow, asking for help in the team and asking AI?

Believing I could answer this well in a small reflection section in a random semester project would be ambitious and my confidence in solving world problems casually as part of my school curriculum is not that high. However, as I am expected to show growth in this section, and I am not planning on uploading the history of my personal scale, I would summarize my way of overcoming this and dealing with the situation in a single quote - "talk with people". It does not matter if this is about seeking supervision, lacking motivation to open the project rather than scrolling through social media or just generally feeling lost in the project. Almost always the answer is to talk with people, more is usually better.

Another challenge was the lack of a good structure and a highly related problem of a lack of transparency. The scrum teams were allowed to choose their own ways of working and they did. Frontend managed via Jira, IoT relied on Trello, MAL got immersed in Figma. On one hand - scrum utopia; every team choosing their own personal efficient way of working. On the other hand - a complete mess and a lack of transparency. The confusion about what is doing what, mostly across teams and the amount of accusations and paranoia that arised from that is unprecedented. I once

completed a course about remote team leadership by Chris Croft, and among many of the takeaways was the importance of transparency and the feeling of inclusion in the project. What a mistake of not treating this project as a remote team. I wish I could have made everyone feel more included and more onboard about what is going on. Alignment meetings and a sea of DMs was not enough.

Yet another beautiful challenge I had to keep for the end was the problem with getting everyone onboard for specific technologies. I personally had my VPS with Coolify installed as a sort of "hammer for all nails" solution. I knew it might have not been the best but nobody else had better ideas available and I saw that even going with Azure, AWS, … would have a similar learning curve, this time with what seemed to be even less people knowing what is going on. I saved this challenge for the end to leave on a more positive note as I believe we have managed this well. People seemed to have researched about Coolify, everyone was given admin access since the beginning and every time I wasn't available, various people decided to fix things on their own rather than to wait for me. And this was exactly what I wanted it to be. I am far from calling this perfection, but I am satisfied with what I could have influenced and what I have achieved on my side.

Reflection Forward

It is difficult to point out in several small chunks the key takeaways from such a project. On one hand, not having anything to point out suggests that a proper reflection has not been done, and on the other, allowing myself to summarize my experience into bullet points would be a massive oversimplification. However, these are my strongest points to be considered in future projects:

Being data-driven inwards, not just outwards - on the previous semester project I was able to work in a highly data-driven fashion and I considered it the peak of what can be achieved in terms of acting rationally on such projects. In the future, I will focus on not just scanning the business aspects of features, and similar but also to understand better the team itself, skills, capabilities, obstacles, and so on.

Talk to people - I have very strongly outlined the idea behind this above but again, the best solution for uncertainty, and overall "human problems" is to talk to "humans". Seeking supervision, understanding the situation beyond the project, aligning, and so on.

Being remote is not about the distance - Regardless of how inaccurate this point sounds, yes, sometimes even though we are in the same class, we communicate in highly asynchronous ways and as it is usual with Software projects, a lot is managed digitally. I will treat more projects in this way as disregarding the specific aspects of such projects can be problematic and the gap will reveal itself in nasty ways.

6. REFLECTION ON SUPERVISION

6.1 The Supervision Process

[Describe how supervision was conducted. How often did you meet? How did you prepare for meetings? What format did sessions typically take?]

6.1.1 MAL

Although we found brief moments of asking for advice in the beginning, around the elaboration phase we missed guidance that we could have sought. Towards the end, we set a goal of meeting every 1-2 weeks to never go off the path too much. Overall, we could have started earlier with asking for supervisor feedback and could have easily had less meetings towards the end if we had the correct trajectory from the start.

6.2 Impact on the Project

[In what specific ways did supervision change or improve the project? Were there suggestions or challenges from the supervisor that redirected your work? Were there times you disagreed with feedback — how did you handle that?]

6.2.1 MAL

The feedback radically changed and shaped the final approaches in the project. Not only did the advice mostly turn out to be true (predicting IoT data may be insufficient, mentioning which features could hinder or improve accuracy, etc.) but it also made us think about the problem in a more structured way. We had to justify our decisions and approaches to the supervisor, which forced us to be more critical of our own work and assumptions. This was a very valuable process that we could have benefited from even more if we had started it earlier.

6.3 Lessons for Future Projects

[What will you do differently in your next supervised project? Examples: prepare more structured agendas, present partial results earlier, ask more targeted questions, be more proactive about seeking feedback. Be concrete and actionable.]

As often, the main lesson is to be more proactive about seeking feedback and perhaps searching for efficient middle ground in many aspects. We correctly identified meetings of all to be inefficient and pointless, however, we suffered from alignment throughout the whole project. We sought supervision towards the end and missed it a lot in the middle. Overall, having a more structured start could have been of most advantage, it must, however, be noted that the semester project required utilizing the knowledge from other subjects which in the beginning made it difficult to have an accurate overview of the tasks to be accomplished and the approaches to be taken. We also had a lot of "silence and pointing fingers", which is expected of large groups, where everyone expects someone else to bear the responsibility. Better defined roles and responsibilities as well as more structure and alignment could have fixed this problem and made the project more efficient and enjoyable.

7. CONCLUSION

7.1 Group Summary

[Summarise the group's overall experience. What defined this project's process? What are the most important things you collectively learned — about software engineering practice, teamwork, or the PBL learning model? How has this project prepared you for professional work?]

7.2 Recommendations

[Produce a practical list of do's and don'ts for future groups undertaking a similar project. These should be grounded in your actual experience — not generic advice.

Do:

- seek supervision early and often
- establish clear communication channels and regular check-ins
- define roles and responsibilities clearly from the start
- set up CI/CD pipelines early in the development process
- use tools to track progress and accountability

Avoid:

- leaving integration until the end
- neglecting testing until late in the project
- allowing motivation to drop without addressing it
- letting conflict fester without resolution
- assuming everyone is on the same page without regular alignment checks

8. REFERENCES

- Vroom, V. H. (1964). Work and motivation .
- Darley, J. M., & Latane, B. (1968). Bystander intervention in emergencies: Diffusion of responsibility. Journal of Personality and Social Psychology , 8 (4, Pt.1), 377–383. **https://doi.org/10.1037/h0025589**
-